Anatomía de un LLM

Fundamentos matemáticos de los modelos de lenguaje

Fase II · Aprendizaje automático y redes neuronales

Módulo 11

Entrenar redes profundas

Resumen del módulo

Edición de octubre de 2026

Sobre este resumen

Este documento reúne lo indispensable de los apuntes del Módulo 11: el texto expositivo, todas las definiciones y todos los resultados (teoremas, proposiciones, lemas y corolarios), las tablas y las observaciones de contenido. Se han suprimido las demostraciones, los ejemplos, las lecturas, las figuras, los ejercicios y sus soluciones. La numeración de temas, secciones, enunciados y ecuaciones es la misma que en los apuntes, de modo que cualquier referencia puede buscarse allí; las que remiten a elementos suprimidos (un ejemplo, una figura, una ecuación de una demostración) conservan su número en los apuntes.

Notación

Símbolo	Significado
$h_\ell \in \mathbb{R}^d$, $z_\ell = W_\ell h_{\ell-1}$	activación y preactivación de la capa ℓ, con $\ell = 1, \dots, L$
$J_\ell = \partial h_\ell / \partial h_{\ell-1}$	jacobiano de la capa (o de la rama residual, tema 2)
$\mathcal{L}$, $g = \nabla_\theta \mathcal{L}$	pérdida y gradiente respecto de los parámetros
$\|A\|_2$, $\sigma_{\min}(A)$	norma espectral (mayor valor singular) y menor valor singular
Var, $\mathbb{E}$	varianza y esperanza
μ, σ^2, $\mathrm{RMS}(x)$	media, varianza y raíz cuadrática media de un vector
γ, β; g	ganancia y sesgo de LayerNorm; ganancia de RMSNorm
ϵ	constante de estabilidad dentro de una raíz
p, $m_i \sim \mathrm{Bernoulli}(1-p)$	probabilidad de apagado del dropout y máscara
c	umbral del recorte del gradiente
e, m	bits de exponente y de mantisa de un formato de coma flotante
$\varepsilon = 2^{-m}$, $u = \varepsilon/2$	épsilon de máquina y unidad de redondeo
$\mathrm{fl}(x)$	número representable más cercano a x
$\mathrm{LSE}(z)$	log-sum-exp, $\ln \sum_i e^{z_i}$
$u \odot w$, $\mathbf{1}$	producto elemento a elemento; vector de unos

Índice

Sobre este resumen **ii**

1 Gradientes que se desvanecen o explotan **1**
1.1 El gradiente como producto de jacobianos . . . 1
1.2 Cotas mediante valores singulares . . . 1
1.3 Saturación de las activaciones . . . 2
1.4 Propagación de la varianza e inicialización . . . 3

2 Conexiones residuales **4**
2.1 Clases de funciones anidadas . . . 4
2.2 Propagación en una red residual . . . 4
2.3 El flujo residual de un Transformer . . . 5

3 Normalización: LayerNorm, RMSNorm y pre-norm **6**
3.1 Normalización por lotes y por capa . . . 6
3.2 Invariancias . . . 7
3.3 Post-norm y pre-norm . . . 7

4 Dropout y recorte del gradiente **9**
4.1 Dropout . . . 9
4.2 Recorte del gradiente por norma global . . . 10

5 Aritmética de coma flotante y precisión mixta **11**
5.1 Formatos de coma flotante . . . 11
5.2 Redondeo y unidad de redondeo . . . 12
5.3 Precisión mixta . . . 13

6 Estabilidad numérica de softmax y entropía cruzada **14**
6.1 Límites de la exponencial . . . 14
6.2 La función log-sum-exp . . . 14
6.3 Entropía cruzada a partir de los logits . . . 15
6.4 Propagación de `NaN` y diagnóstico . . . 15

Bibliografía **17**

Tema 1

Gradientes que se desvanecen o explotan

En una red profunda, el gradiente que recibe una capa es el producto de los jacobianos de todas las capas que tiene por encima. Este tema formaliza esa observación, acota la norma del producto mediante valores singulares y deduce de ella los dos fenómenos que dificultan el entrenamiento en profundidad: el desvanecimiento y la explosión del gradiente. Se estudian después sus dos causas principales, la escala de los pesos y la saturación de las activaciones, y se obtiene la inicialización que mantiene constante la varianza de la señal. Se sigue [9, §8.2.5, §10.7], [26, §5.4 y §9.7] y los artículos de Pascanu, Mikolov y Bengio [18], Glorot y Bengio [7] y He *et al.* [11].

1.1 El gradiente como producto de jacobianos

Definición 1.1 (Red profunda de anchura constante). Sea $L \geq 1$. Una *red de L capas* con anchura d y activación $f\colon \mathbb{R} \to \mathbb{R}$ derivable transforma una entrada $h_0 \in \mathbb{R}^d$ mediante

$$z_\ell = W_\ell h_{\ell-1}, \qquad h_\ell = f(z_\ell), \qquad \ell = 1, \dots, L,$$

con $W_\ell \in \mathbb{R}^{d\times d}$ y f aplicada componente a componente. La pérdida $\mathcal{L}$ depende de la entrada solo a través de h_L. El *jacobiano de la capa ℓ* es

$$J_\ell = \frac{\partial h_\ell}{\partial h_{\ell-1}} = \operatorname{diag}\big(f'(z_\ell)\big)\, W_\ell.$$

Los sesgos se omiten porque no intervienen en los jacobianos respecto de las activaciones. La anchura constante simplifica la notación; todos los resultados se extienden a anchuras variables.

Proposición 1.2 (Gradiente en profundidad). *Con la notación de la definición 1.1, escribiendo los gradientes como vectores columna,*

$$\nabla_{h_\ell}\mathcal{L} = J_{\ell+1}^\top J_{\ell+2}^\top \cdots J_L^\top \, \nabla_{h_L}\mathcal{L}, \qquad \frac{\partial \mathcal{L}}{\partial W_\ell} = \Big(\operatorname{diag}\big(f'(z_\ell)\big)\nabla_{h_\ell}\mathcal{L}\Big)\, h_{\ell-1}^\top.$$

El gradiente de los pesos de la capa ℓ contiene, por tanto, el producto de los $L-\ell$ jacobianos que tiene encima. Las primeras capas atraviesan casi todos.

1.2 Cotas mediante valores singulares

Para matrices, el papel del factor a lo desempeñan los valores singulares. Para $A \in \mathbb{R}^{d\times d}$ se escribe $\|A\|_2 = \max_{\|x\|=1}\|Ax\|$ (el mayor valor singular, módulo 3) y $\sigma_{\min}(A) = \min_{\|x\|=1}\|Ax\|$ (el menor). Ambos coinciden para A y $A^\top$.

Factor por capa a	$L = 10$	$L = 24$	Efecto
0,25	$9{,}5 \cdot 10^{-7}$	$3{,}6 \cdot 10^{-15}$	desvanecimiento extremo
0,5	$9{,}8 \cdot 10^{-4}$	$6{,}0 \cdot 10^{-8}$	las primeras capas no aprenden
1	1	1	caso ideal
1,1	2,59	9,85	crecimiento moderado
1,5	57,7	$1{,}7 \cdot 10^{4}$	explosión

Cuadro 1.1: Factor a^L por el que se multiplica el gradiente al atravesar L capas escalares.

Lema 1.4. *Para $A, B \in \mathbb{R}^{d\times d}$ se cumple $\sigma_{\min}(A)\,\sigma_{\min}(B) \le \sigma_{\min}(AB) \le \|AB\|_2 \le \|A\|_2\|B\|_2$.*

Teorema 1.5 (Cotas del gradiente en profundidad). *Con la notación de la definición 1.1,*

$$\prod_{\ell=1}^{L} \sigma_{\min}(J_\ell) \;\le\; \frac{\|\nabla_{h_0}\mathcal{L}\|}{\|\nabla_{h_L}\mathcal{L}\|} \;\le\; \prod_{\ell=1}^{L} \|J_\ell\|_2 \;\le\; \prod_{\ell=1}^{L} \gamma\,\|W_\ell\|_2, \qquad \gamma = \sup_{t\in\mathbb{R}} |f'(t)|.$$

Corolario 1.6 (Condición suficiente de desvanecimiento). *Si existe $\lambda < 1$ tal que $\gamma\|W_\ell\|_2 \le \lambda$ para todo ℓ, entonces $\|\nabla_{h_0}\mathcal{L}\| \le \lambda^L \|\nabla_{h_L}\mathcal{L}\|$, que tiende a 0 exponencialmente con L. En consecuencia, que el gradiente explote exige que $\|W_\ell\|_2 > 1/\gamma$ para alguna capa.*

En una red recurrente la misma matriz W se aplica una vez por paso de tiempo, de modo que el corolario se lee con L igual a la longitud de la secuencia. Es el resultado de [18, §2.1]: con $\gamma = 1$ para tanh y $\gamma = 1/4$ para la sigmoide logística, basta $\|W\|_2 < 1/\gamma$ para que el gradiente se desvanezca a lo largo del tiempo. Goodfellow *et al.* [9, §10.7] y Zhang *et al.* [26, §9.7.2] presentan el caso lineal, en el que el gradiente contiene las potencias W^t.

Observación 1.8. Los productos de muchos factores grandes crean en la superficie de pérdida regiones casi verticales, llamadas *acantilados* [9, §8.2.4]. Un paso de gradiente en una de ellas puede expulsar los parámetros lejos de la región que se estaba explorando. El recorte del gradiente del tema 4 es el remedio estándar.

1.3 Saturación de las activaciones

El factor $\operatorname{diag}(f'(z_\ell))$ del jacobiano depende de las preactivaciones. Para las activaciones del módulo 9:

- $\tanh'(t) = 1 - \tanh^2(t) \in (0, 1]$, con máximo en $t = 0$. Vale 0,420 en $t = 1$, 0,0707 en $t = 2$ y 0,0099 en $t = 3$.
- La sigmoide $s(t) = 1/(1 + e^{-t})$ cumple $s'(t) = s(t)(1 - s(t)) \le 1/4$.
- $\mathrm{ReLU}'(t) = 1$ si $t > 0$ y 0 si $t < 0$.

Una unidad con $|z|$ grande está *saturada*: su derivada es casi nula y anula la fila correspondiente del jacobiano. Si una unidad ReLU tiene preactivación negativa para todas las entradas, su gradiente es exactamente cero y deja de aprender. Por el teorema 1.5, basta que una sola capa tenga $\sigma_{\min}(J_\ell)$ pequeño para que el producto completo pueda serlo.

1.4 Propagación de la varianza e inicialización

La escala de los pesos se elige para que la varianza de la señal no crezca ni decrezca de una capa a la siguiente. El resultado clave es el siguiente [26, §5.4.2].

Teorema 1.9 (Propagación de la varianza). *Sea* $y = \sum_{i=1}^{n} w_i x_i$*, donde las* $2n$ *variables* $w_1, \dots, w_n, x_1, \dots, x_n$ *son independientes,* $\mathbb{E}[w_i] = 0$*,* $\operatorname{Var}(w_i) = \sigma^2$ *y* $\mathbb{E}[x_i^2] = q$ *para todo* i*. Entonces* $\mathbb{E}[y] = 0$ *y*

$$\operatorname{Var}(y) = n\,\sigma^2\,q.$$

Si además $\mathbb{E}[x_i] = 0$*, entonces* $q = \operatorname{Var}(x_i)$*.*

Corolario 1.10 (Inicialización de Xavier). *En una red lineal* ($f(t) = t$) *con entradas centradas,* $\operatorname{Var}(z_\ell) = (n\sigma^2)^\ell \operatorname{Var}(h_0)$*. La varianza se conserva en todas las capas si y solo si* $\sigma^2 = 1/n$*. La pasada hacia atrás multiplica por las matrices traspuestas, cuyo número de entradas por fila es el de salidas, y la varianza del gradiente se conserva con* $\sigma^2 = 1/n_{sal}$*; Glorot y Bengio [7] proponen el compromiso* $\sigma^2 = 2/(n_{ent} + n_{sal})$*.*

Proposición 1.11 (Inicialización de He). *Sea* z *una variable aleatoria con densidad simétrica respecto de 0 y varianza* v*. Entonces* $\mathbb{E}[\mathrm{ReLU}(z)^2] = v/2$*. En consecuencia, en una red ReLU la varianza de las preactivaciones se conserva con* $\sigma^2 = 2/n$ *[11].*

Tema 2

Conexiones residuales

Una conexión residual suma a la salida de un bloque su propia entrada. Con este cambio mínimo, el jacobiano de cada bloque pasa a ser la identidad más una corrección, y el producto de jacobianos del tema 1 contiene siempre un término que no se atenúa con la profundidad. Este tema define las redes residuales, demuestra sus propiedades de propagación hacia delante y hacia atrás, y describe el flujo residual de un Transformer. Se sigue el artículo original de He *et al.* [10], su continuación sobre aplicaciones identidad [12], [26, §8.6] y [2, §9.5].

2.1 Clases de funciones anidadas

Zhang *et al.* [26, §8.6.1] motivan las redes residuales con un argumento sobre clases de funciones. Sea $\mathcal{F}_L$ el conjunto de funciones que puede representar una arquitectura de L bloques. Al añadir un bloque se esperaría $\mathcal{F}_L \subseteq \mathcal{F}_{L+1}$: una red más profunda debería poder imitar a la menos profunda. En una red plana esto exige que el bloque nuevo aprenda exactamente la identidad, lo que no es fácil con capas no lineales; empíricamente, las redes planas muy profundas obtienen *mayor* error de entrenamiento que otras menos profundas [10].

Definición 2.1 (Bloque y red residual). Un *bloque residual* es una aplicación $\mathbb{R}^d \to \mathbb{R}^d$ de la forma

$$h \longmapsto h + F(h;\theta),$$

donde F, la *rama residual*, es una red diferenciable con parámetros θ. Una *red residual* de L bloques calcula $h_\ell = h_{\ell-1} + F_\ell(h_{\ell-1})$ para $\ell = 1, \dots, L$.

Proposición 2.2 (Anidamiento). *Si cada rama F_ℓ termina en una capa lineal $u \mapsto Vu$, la red residual de $L+1$ bloques puede representar cualquier función que represente la de L bloques.*

La proposición explica también la idea de *residuo*: el bloque no tiene que aprender la transformación deseada $H(h)$, sino la diferencia $F(h) = H(h) - h$, que es pequeña cuando H se parece a la identidad.

2.2 Propagación en una red residual

Proposición 2.3 (Propagación hacia delante). *En una red residual, para $0 \le \ell < L$,*

$$h_L = h_\ell + \sum_{k=\ell+1}^{L} F_k(h_{k-1}).$$

La salida es la entrada de cualquier bloque más la suma de las aportaciones de los bloques posteriores. Esta lectura aditiva, destacada en [12, §2], contrasta con la composición $f_L \circ \cdots \circ f_1$ de una red plana.

Teorema 2.4 (Jacobiano de una red residual). *Sea $J_\ell = \partial F_\ell / \partial h_{\ell-1}$, evaluado en $h_{\ell-1}$. Entonces*

$$\frac{\partial h_L}{\partial h_0} = (I + J_L)(I + J_{L-1}) \cdots (I + J_1) = I + \sum_{\ell=1}^{L} J_\ell + \sum_{1 \le \ell < k \le L} J_k J_\ell + \cdots + J_L \cdots J_1,$$

donde el desarrollo tiene 2^L términos, uno por cada subconjunto de bloques, con los factores en orden decreciente de índice. En particular, contiene el término I.

Corolario 2.5. $\nabla_{h_0} \mathcal{L} = \nabla_{h_L} \mathcal{L} + \left(\sum_\ell J_\ell + \sum_{\ell<k} J_k J_\ell + \cdots \right)^\top \nabla_{h_L} \mathcal{L}$. *El gradiente de la salida llega a la entrada sin atenuar, sumado a correcciones.*

Cada término del desarrollo corresponde a un camino por la red que atraviesa un subconjunto de ramas y salta las demás. Una red plana solo dispone del camino $J_L \cdots J_1$, que es el último término.

Proposición 2.7 (Cota inferior). *Si $\|J_\ell\|_2 \le a < 1$ para todo ℓ, entonces*

$$\sigma_{\min}\left(\frac{\partial h_L}{\partial h_0}\right) \ge (1 - a)^L > 0.$$

En particular, ningún vector de gradiente no nulo en la salida se anula en la entrada.

En una red plana, en cambio, basta un J_ℓ con núcleo no trivial (una capa ReLU con unidades apagadas, por ejemplo) para que algún gradiente se anule. La cota de la proposición 2.7 no impide el desvanecimiento si a es grande, pero al inicio del entrenamiento las ramas tienen jacobianos pequeños y $(1 \pm a)^L$ queda cerca de 1: con $a = 0{,}01$ y $L = 24$, entre $0{,}99^{24} = 0{,}786$ y $1{,}01^{24} = 1{,}27$.

2.3 El flujo residual de un Transformer

En un Transformer, el vector $h_\ell \in \mathbb{R}^d$ asociado a cada token recorre todos los bloques. Cada subbloque de atención o de perceptrón multicapa lee de él y le suma su aportación; por eso se le llama *flujo residual*. La proposición 2.3 dice que el flujo de la última capa es la suma del embedding inicial y de todas las aportaciones. En el módulo 23 se usará esta descomposición para interpretar qué escribe cada bloque.

Proposición 2.8 (Crecimiento del flujo residual). *Supóngase que, al inicio del entrenamiento, las aportaciones $F_k(h_{k-1})$ de una componente del flujo son variables aleatorias de media nula, incorreladas entre sí y con h_0, y con varianza v cada una. Entonces* $\mathrm{Var}\big((h_L)_i\big) = \mathrm{Var}\big((h_0)_i\big) + Lv$*: la desviación típica del flujo crece como $\sqrt{L}$.*

Este crecimiento motiva dos prácticas de los modelos de lenguaje. GPT-2 [19, §2.3] multiplica en la inicialización los pesos de las capas que escriben en el flujo por $1/\sqrt{N}$, con N el número de capas residuales, lo que reduce cada varianza v en un factor N. Y los Transformer pre-norm añaden una normalización final antes de la capa de salida (tema 3).

Tema 3

Normalización: LayerNorm, RMSNorm y pre-norm

Normalizar una capa consiste en reescalar cada vector de activaciones con estadísticos calculados sobre él mismo, de modo que su escala no derive de una capa a otra. Este tema define la normalización por lotes, la normalización por capa (LayerNorm) y su simplificación RMSNorm, demuestra sus propiedades de invariancia, calcula el jacobiano de RMSNorm y analiza dónde debe colocarse la normalización respecto de la conexión residual. Se sigue [2, §7.4], [26, §8.5] y [9, §8.7.1], junto con los artículos de Ioffe y Szegedy [15], Ba, Kiros y Hinton [1], Zhang y Sennrich [27] y Xiong *et al.* [25].

3.1 Normalización por lotes y por capa

Definición 3.1 (Normalización por lotes). Sea un minibatch de B vectores $x^{(1)}, \dots, x^{(B)} \in \mathbb{R}^d$. La *normalización por lotes* (BatchNorm) normaliza cada *componente* i con la media y la varianza de esa componente sobre el lote:

$$\hat{x}_i^{(b)} = \frac{x_i^{(b)} - \mu_i}{\sqrt{\sigma_i^2 + \epsilon}}, \qquad \mu_i = \frac{1}{B}\sum_{b=1}^{B} x_i^{(b)}, \quad \sigma_i^2 = \frac{1}{B}\sum_{b=1}^{B}\big(x_i^{(b)} - \mu_i\big)^2,$$

y después aplica una ganancia y un sesgo aprendidos por componente [15].

La salida de BatchNorm para un ejemplo depende de los demás ejemplos del lote; en inferencia se sustituyen μ_i y σ_i^2 por medias móviles estimadas durante el entrenamiento [26, §8.5.2]. Para secuencias de longitud variable y para la generación token a token, esta dependencia es un inconveniente. La normalización por capa la elimina normalizando sobre las componentes de cada vector.

Definición 3.2 (LayerNorm). Sea $x \in \mathbb{R}^d$, y sean $\gamma, \beta \in \mathbb{R}^d$ parámetros y $\epsilon \geq 0$. Con

$$\mu = \frac{1}{d}\sum_{i=1}^{d} x_i, \qquad \sigma^2 = \frac{1}{d}\sum_{i=1}^{d}(x_i - \mu)^2,$$

la *normalización por capa* es

$$\mathrm{LN}(x)_i = \gamma_i \, \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta_i, \qquad i = 1, \dots, d \ [1].$$

Definición 3.3 (RMSNorm). Sea $x \in \mathbb{R}^d$, $g \in \mathbb{R}^d$ y $\epsilon \geq 0$. Con $\mathrm{RMS}(x) = \sqrt{\frac{1}{d}\sum_{i=1}^{d} x_i^2 + \epsilon}$, se define

$$\mathrm{RMSNorm}(x)_i = g_i \, \frac{x_i}{\mathrm{RMS}(x)}, \qquad i = 1, \dots, d \ [27].$$

RMSNorm suprime el centrado y el sesgo β: solo fija la escala. En un Transformer ambas se aplican por separado al vector de cada token, por lo que funcionan igual en entrenamiento y en inferencia y con lotes de cualquier tamaño.

Proposición 3.5 (Estadísticos de la salida). *Sea $\epsilon = 0$, $\gamma = g = \mathbf{1}$ y $\beta = 0$.*

(i) Si x no es constante, $y = \mathrm{LN}(x)$ cumple $\frac{1}{d}\sum_i y_i = 0$ y $\frac{1}{d}\sum_i y_i^2 = 1$.

(ii) Si $x \neq 0$, $y = \mathrm{RMSNorm}(x)$ cumple $\frac{1}{d}\sum_i y_i^2 = 1$, es decir, $\|y\| = \sqrt{d}$.

3.2 Invariancias

Proposición 3.6 (Invariancias). *Sea $\epsilon = 0$, $\alpha > 0$ y $c \in \mathbb{R}$.*

(i) $\mathrm{LN}(\alpha x + c\mathbf{1}) = \mathrm{LN}(x)$ *para todo x no constante.*

(ii) $\mathrm{RMSNorm}(\alpha x) = \mathrm{RMSNorm}(x)$ *para todo $x \neq 0$, pero en general* $\mathrm{RMSNorm}(x + c\mathbf{1}) \neq \mathrm{RMSNorm}(x)$.

(iii) Si la entrada de la normalización es $x = Wu$ con $W \in \mathbb{R}^{d\times n}$, la salida no cambia al sustituir W por αW, para LayerNorm y para RMSNorm.

La propiedad (iii) figura en la tabla de invariancias de Ba *et al.* [1, §5.1]: la salida no depende de la escala de los pesos de la capa anterior. Zhang y Sennrich [27] conservan esta invariancia de escala, que consideran la responsable del buen comportamiento de LayerNorm, y renuncian a la de traslación, con lo que se ahorra el cálculo de la media.

Teorema 3.7 (Jacobiano de RMSNorm). *Sea $\epsilon = 0$, $x \neq 0$ y $r = \mathrm{RMS}(x)$. El jacobiano de $y = \mathrm{RMSNorm}(x)$ es*

$$J = \frac{\partial y}{\partial x} = \frac{1}{r}\,\mathrm{diag}(g)\Big(I - \frac{xx^\top}{d\,r^2}\Big) = \frac{1}{r}\,\mathrm{diag}(g)\Big(I - \frac{xx^\top}{\|x\|^2}\Big).$$

En consecuencia, $Jx = 0$ y, para todo $v \in \mathbb{R}^d$, $x^\top(J^\top v) = 0$: el gradiente que atraviesa RMSNorm hacia atrás es ortogonal a x, y su norma es a lo sumo $\max_i|g_i|\,\|v\|/r$.

La identidad $Jx = 0$ es la versión infinitesimal de la invariancia de escala: al derivar $\mathrm{RMSNorm}((1+t)x) = \mathrm{RMSNorm}(x)$ en $t = 0$ se obtiene $Jx = 0$. La cota $\|v\|/r$ muestra el efecto estabilizador: si una capa anterior infla sus activaciones, r crece y el gradiente que se le devuelve se reduce en la misma proporción.

Observación 3.8. Un cálculo análogo, con $\hat{y} = (x - \mu\mathbf{1})/\sigma$, da para LayerNorm con $\gamma = \mathbf{1}$, $\beta = 0$ y $\epsilon = 0$

$$\frac{\partial \hat{y}}{\partial x} = \frac{1}{\sigma}\Big(I - \frac{\mathbf{1}\mathbf{1}^\top}{d} - \frac{\hat{y}\hat{y}^\top}{d}\Big),$$

que anula tanto $\mathbf{1}$ como $x - \mu\mathbf{1}$, de acuerdo con la invariancia (i) de la proposición 3.6.

3.3 Post-norm y pre-norm

Definición 3.9 (Posición de la normalización). Sea Norm una normalización (LayerNorm o RMSNorm) y F una rama residual. Un bloque es

post-norm si $h_\ell = \mathrm{Norm}\big(h_{\ell-1} + F(h_{\ell-1})\big)$, *pre-norm* si $h_\ell = h_{\ell-1} + F\big(\mathrm{Norm}(h_{\ell-1})\big)$.

El Transformer original [23] es post-norm. GPT-2 [19, §2.3] trasladó la normalización a la entrada de cada subbloque y añadió una normalización final tras el último bloque; LLaMA [22] usa pre-norm con RMSNorm, y la mayoría de los modelos abiertos posteriores la han adoptado.

Proposición 3.10 (Jacobianos de ambas posiciones). *Sea N_ℓ el jacobiano de la normalización del bloque ℓ y J_ℓ el de la rama F_ℓ, cada uno evaluado en su argumento. Entonces*

$$\textit{post-norm: } \frac{\partial h_L}{\partial h_0} = \prod_{\ell=L}^{1} N_\ell \,(I + J_\ell), \qquad \textit{pre-norm: } \frac{\partial h_L}{\partial h_0} = \prod_{\ell=L}^{1} \big(I + J_\ell N_\ell\big),$$

donde los productos se escriben con los índices en orden decreciente. Solo el desarrollo de pre-norm contiene el término I.

En pre-norm, el flujo residual no pasa por ninguna normalización y conserva el camino identidad. A cambio, su norma crece con la profundidad (proposición 2.8), y hace falta una normalización final antes de la proyección a logits. Xiong *et al.* [25] demuestran, con argumentos de campo medio, que en post-norm el gradiente esperado de los parámetros próximos a la salida es grande en la inicialización, lo que obliga a un calentamiento largo de la tasa de aprendizaje (módulo 7); en pre-norm los gradientes están acotados y el calentamiento puede acortarse o suprimirse.

Observación 3.11 (Sobre los modelos comerciales). Anthropic y Google no han hecho pública la normalización que usan Claude y Gemini. El código publicado de Grok-1 [24] incluye capas RMSNorm.

Tema 4

Dropout y recorte del gradiente

Este tema reúne dos técnicas que actúan sobre el entrenamiento sin cambiar la arquitectura. El dropout introduce ruido multiplicativo en las activaciones para regularizar; el recorte del gradiente limita la longitud de cada paso para que un gradiente anómalo no destruya lo aprendido. Se demuestran las propiedades que justifican ambas: el dropout invertido es insesgado y su varianza se calcula en forma cerrada; el recorte por norma global conserva la dirección del gradiente y acota el paso. Se sigue [9, §7.12], [26, §5.6], [2, §9.6.1] y las notas de CS231n [3] para el dropout, con el artículo de Srivastava *et al.* [21]; y [9, §8.2.4 y §10.11] y el artículo de Pascanu, Mikolov y Bengio [18] para el recorte.

4.1 Dropout

Definición 4.1 (Dropout invertido). Sea $h \in \mathbb{R}^d$ un vector de activaciones y $p \in [0, 1)$. Durante el entrenamiento, el *dropout invertido* con probabilidad p sustituye h por

$$\tilde{h}_i = \frac{m_i \, h_i}{1 - p}, \qquad m_i \sim \text{Bernoulli}(1 - p) \text{ independientes}, \qquad i = 1, \dots, d.$$

En evaluación se usa $\tilde{h} = h$.

Cada unidad se apaga con probabilidad p y las que sobreviven se amplifican por $1/(1-p)$. La variante original [21] no amplificaba durante el entrenamiento y, en cambio, multiplicaba los pesos por $1 - p$ en la evaluación; la variante invertida traslada esa corrección al entrenamiento y deja intacto el código de inferencia [3].

Proposición 4.2 (Esperanza y varianza). *Con la notación de la definición 4.1, para h fijo,*

$$\mathbb{E}[\tilde{h}_i] = h_i, \qquad \operatorname{Var}(\tilde{h}_i) = \frac{p}{1 - p} \, h_i^2.$$

Corolario 4.3 (Dropout antes de una capa lineal). *Sea $y = w^\top \tilde{h}$ con $w \in \mathbb{R}^d$ fijo. Entonces $\mathbb{E}[y] = w^\top h$ y $\operatorname{Var}(y) = \frac{p}{1-p} \sum_{i=1}^{d} w_i^2 h_i^2$.*

El corolario muestra que, para una capa lineal, evaluar sin máscara da exactamente la salida media sobre todas las máscaras. Goodfellow *et al.* [9, §7.12] interpretan el dropout como el entrenamiento simultáneo de los 2^d subredes que se obtienen al eliminar unidades, con parámetros compartidos; la regla de evaluación sin máscara, llamada *regla de escalado de pesos*, es exacta para modelos sin no linealidades ocultas, como el anterior, y solo aproximada en redes profundas no lineales.

4.2 Recorte del gradiente por norma global

Definición 4.6 (Recorte por norma global). Sean $g^{(1)}, \dots, g^{(K)}$ los gradientes de los K tensores de parámetros de un modelo y sea $g \in \mathbb{R}^n$ su concatenación. Dado un umbral $c > 0$, el *recorte por norma global* sustituye cada $g^{(k)}$ por

$$\hat{g}^{(k)} = \alpha\, g^{(k)}, \qquad \alpha = \min\Big(1,\ \frac{c}{\|g\|_2}\Big),$$

con el convenio $\alpha = 1$ si $g = 0$.

Proposición 4.7 (Propiedades del recorte). *Sea $\hat{g} = \alpha g$ el gradiente recortado.*

(i) $\hat{g}$ tiene la dirección y el sentido de g, y $\|\hat{g}\|_2 = \min(\|g\|_2, c)$.

(ii) Un paso de descenso $\theta \leftarrow \theta - \eta\hat{g}$ cumple $\|\Delta\theta\|_2 \le \eta c$.

(iii) Si $\hat{g} \neq 0$, $\hat{g}$ es una dirección de descenso: $\langle \nabla\mathcal{L}, -\hat{g}\rangle < 0$ cuando $\hat{g}$ se calcula con el gradiente exacto.

(iv) Recortar cada tensor por separado con el mismo umbral conserva, en general, la dirección de cada $g^{(k)}$ pero no la de g.

El recorte lo propusieron Pascanu, Mikolov y Bengio [18, §3.2] para redes recurrentes, donde el corolario 1.6 permite gradientes que explotan; en un acantilado de la superficie de pérdida [9, §8.2.4], el gradiente apunta en una dirección razonable pero tiene una longitud desproporcionada, y el recorte conserva lo primero y corrige lo segundo. En PyTorch se aplica con `torch.nn.utils.clip_grad_norm_` entre `backward()` y el paso del optimizador.

Tema 5

Aritmética de coma flotante y precisión mixta

Los modelos de lenguaje se entrenan con números de 16 e incluso de 8 bits. Un formato de coma flotante reparte sus bits entre el exponente, que fija el rango, y la mantisa, que fija la precisión relativa; esa elección determina qué valores desbordan, cuáles se anulan y qué sumandos se pierden al acumular. Este tema define los formatos con la norma IEEE 754 [14], demuestra las cotas de rango y de error de redondeo, estudia la absorción de sumandos pequeños y justifica con ella los tres ingredientes de la precisión mixta de Micikevicius *et al.* [16]. Se sigue además el tutorial de Goldberg [8] y el libro de Higham [13].

5.1 Formatos de coma flotante

Definición 5.1 (Formato binario de coma flotante). Sean $e \geq 2$ y $m \geq 1$ enteros. Un *formato binario* con e bits de exponente y m bits de mantisa representa cada número con $1 + e + m$ bits: un signo $s \in \{0, 1\}$, un exponente almacenado $E \in \{0, \ldots, 2^e - 1\}$ y una fracción $f = (f_1 \ldots f_m)_2$. Con el *sesgo* $b = 2^{e-1} - 1$:

(i) si $1 \leq E \leq 2^e - 2$, el número es *normal* y vale $x = (-1)^s \, (1.f_1 \ldots f_m)_2 \cdot 2^{E-b}$;

(ii) si $E = 0$, vale $(-1)^s \, (0.f_1 \ldots f_m)_2 \cdot 2^{1-b}$: el cero si $f = 0$ y un número *subnormal* si $f \neq 0$;

(iii) si $E = 2^e - 1$, representa $\pm\infty$ si $f = 0$ y `NaN` (*not a number*) si $f \neq 0$.

Los formatos binary32 (fp32, $e = 8$, $m = 23$) y binary16 (fp16, $e = 5$, $m = 10$) son los de la norma [14]. El formato bfloat16 (bf16, $e = 8$, $m = 7$) no forma parte de ella; se obtiene conservando los 16 bits más significativos de un fp32.

Proposición 5.2 (Rango y precisión). *En el formato de la definición 5.1:*

(i) el mayor número finito es $x_{\max} = (2 - 2^{-m})\, 2^b$*;*

(ii) el menor normal positivo es $x_{\min} = 2^{1-b}$*, y el menor subnormal positivo,* 2^{1-b-m}*;*

(iii) en cada intervalo $[2^k, 2^{k+1})$ *con* $1 - b \leq k \leq b$*, los números representables están equiespaciados con separación* 2^{k-m}*;*

(iv) el épsilon de máquina*, distancia de 1 al siguiente número representable, es* $\varepsilon = 2^{-m}$*.*

Observación 5.4 (El formato E4M3). Los dos formatos de 8 bits proceden de Micikevicius *et al.* [17]. E5M2 sigue las reglas IEEE. E4M3 renuncia a los infinitos y reserva para `NaN` un único patrón de bits (exponente y fracción con todos los bits a 1). Así, el exponente $E = 15$ queda disponible para números normales y el máximo es $(1{,}110)_2 \cdot 2^{15-7} = 1{,}75 \cdot 2^8 = 448$, en lugar del valor $1{,}875 \cdot 2^7 = 240$ que darían las reglas IEEE.

Formato	e	m	$x_{\max}$	$x_{\min}$ (normal)	$\varepsilon = 2^{-m}$	Cifras decimales
fp32	8	23	$3{,}40 \cdot 10^{38}$	$1{,}18 \cdot 10^{-38}$	$1{,}19 \cdot 10^{-7}$	≈ 7
fp16	5	10	65 504	$6{,}10 \cdot 10^{-5}$	$9{,}77 \cdot 10^{-4}$	≈ 3
bf16	8	7	$3{,}39 \cdot 10^{38}$	$1{,}18 \cdot 10^{-38}$	$7{,}81 \cdot 10^{-3}$	≈ 2
fp8 E4M3	4	3	448	$1{,}56 \cdot 10^{-2}$	0,125	≈ 1
fp8 E5M2	5	2	57 344	$6{,}10 \cdot 10^{-5}$	0,25	< 1

Cuadro 5.1: Formatos usados en el entrenamiento de redes. El formato E4M3 no sigue la definición 5.1(iii); véase la observación 5.4.

El formato bf16 tiene el exponente de fp32 y, por tanto, prácticamente su mismo rango, con una precisión mucho menor. Pasar de fp32 a bf16 equivale a redondear la mantisa a 7 bits. Por su rango, bf16 es el formato habitual para entrenar modelos de lenguaje: casi nunca desborda.

5.2 Redondeo y unidad de redondeo

Definición 5.5 (Redondeo al más próximo). Para $x \in \mathbb{R}$ con $|x| \le x_{\max}$, $\mathrm{fl}(x)$ es el número representable más cercano a x; en caso de empate, el de último bit de mantisa par. La *unidad de redondeo* es $u = \varepsilon/2 = 2^{-m-1}$.

Teorema 5.6 (Error relativo de redondeo). *Si $x_{\min} \le |x| \le x_{\max}$, entonces $\mathrm{fl}(x) = x(1+\delta)$ con $|\delta| \le u$.*

La norma IEEE 754 exige que la suma, la resta, el producto, el cociente y la raíz cuadrada se redondeen correctamente: el resultado calculado es $\mathrm{fl}(x \circ y)$, el redondeo del resultado exacto. Junto con el teorema 5.6, esto da el *modelo estándar* del análisis de errores [13, 8]: $\mathrm{fl}(x \circ y) = (x \circ y)(1+\delta)$ con $|\delta| \le u$, mientras no haya desbordamiento ni subdesbordamiento.

Lema 5.7 (Absorción). *Sea $x > 0$ representable con $2^k \le x < 2^{k+1}$ y sea $0 \le y < 2^{k-m-1}$, la mitad de la separación del intervalo. Entonces $\mathrm{fl}(x+y) = x$.*

Corolario 5.8 (Estancamiento de una suma). *Sea $a > 0$ representable y sea la suma recursiva $s_0 = 0$, $s_{j+1} = \mathrm{fl}(s_j + a)$. Si para algún j se cumple $2^k \le s_j < 2^{k+1}$ con $a < 2^{k-m-1}$, entonces $s_i = s_j$ para todo $i \ge j$. En particular, la suma no supera la menor potencia 2^k que cumple $2^k > a\,2^{m+1}$, más una separación.*

Operación	fp32	fp16	bf16
$300 \cdot 300$	90 000	`inf`	90 112
e^{12}	162 754,8	`inf`	162 816
$1 + 2^{-8}$	1,0039	1,0039	1
$1000 + 1$	1001	1001	1000
0,001 sumado 10^4 veces	10,0004	4	0,5

Cuadro 5.2: Resultados de operaciones sencillas en tres formatos, comprobados con PyTorch.

Teorema 5.10 (Error de la suma recursiva). *Sea $\hat{s}_n$ el resultado de sumar $x_1, \dots, x_n$ en orden, $\hat{s}_1 = x_1$, $\hat{s}_k = \mathrm{fl}(\hat{s}_{k-1} + x_k)$, sin desbordamiento ni subdesbordamiento, y sea $(n-1)u < 1$.*

Entonces

$$|\hat{s}_n - \sum_{i=1}^{n} x_i| \le \gamma_{n-1} \sum_{i=1}^{n} |x_i|, \qquad \gamma_k = \frac{ku}{1-ku}.$$

El teorema explica la diferencia entre formatos. En fp32, $u = 2^{-24} = 5{,}96 \cdot 10^{-8}$ y, para $n = 10^4$ sumandos de 0,001, la cota es $\gamma_{9999} \cdot 10 = 0{,}006$; el error observado en la cuadro 5.2 es $4 \cdot 10^{-4}$. En fp16, $u = 2^{-11}$ y $(n-1)u \approx 4{,}9 > 1$: la cota no garantiza nada, y la suma se estanca.

5.3 Precisión mixta

Micikevicius *et al.* [16, §3] entrenan en fp16 sin perder precisión con tres técnicas.

Copia maestra de los pesos en fp32 (§3.1). El optimizador actualiza una copia fp32 de los pesos; la copia fp16 solo se usa en las pasadas hacia delante y hacia atrás. Por el lema 5.7, si un peso $w \in [2^k, 2^{k+1})$ recibe una actualización $|\Delta w| < 2^{k-m-1}$, la suma $w + \Delta w$ redondeada devuelve w. Como $w < 2^{k+1}$, esto ocurre seguro si $|w|/|\Delta w| > 2^{m+2}$ y puede ocurrir desde 2^{m+1}; en fp16 este último cociente es $2^{11} = 2048$, el que señala el artículo.

Escalado de la pérdida (§3.2). Muchos gradientes de activaciones son menores que el menor subnormal de fp16, 2^{-24}, y se anulan. Se multiplica la pérdida por un factor S, se retropropaga y se dividen los gradientes de los pesos por S antes de la actualización, que se omite si aparece un `inf` o un `NaN`. La elección $S = 2^j$ es conveniente por la proposición siguiente.

Proposición 5.11. *Si x y $2^j x$ están en el rango normal, $\mathrm{fl}(2^j x) = 2^j\,\mathrm{fl}(x)$. En particular, multiplicar o dividir un número representable por 2^j es exacto en ausencia de desbordamiento.*

Acumulación en fp32 (§3.3). Los productos escalares de las multiplicaciones de matrices acumulan las sumas parciales en fp32 y solo convierten el resultado a fp16; las reducciones grandes (sumas de la normalización y de la softmax) también se hacen en fp32. El teorema 5.10 justifica esta práctica.

Observación 5.12 (bf16 y fp8). Con bf16 no hace falta escalar la pérdida, porque su rango es el de fp32, pero la copia maestra y la acumulación en fp32 siguen siendo necesarias, porque su precisión es aún menor que la de fp16. Los formatos fp8 se usan en las GPU recientes para las multiplicaciones de matrices, E4M3 en la pasada hacia delante y E5M2 para los gradientes, siempre con factores de escala por tensor [17]; el módulo 17 los estudia con el hardware y el módulo 19 trata la cuantización para inferencia. Anthropic, Google y xAI no han hecho pública la precisión con que entrenan Claude, Gemini y Grok.

Tema 6

Estabilidad numérica de softmax y entropía cruzada

La capa de salida de un modelo de lenguaje calcula una softmax sobre decenas de miles de logits, y la pérdida de entrenamiento es su entropía cruzada (módulo 1). Ambas contienen exponenciales, que en coma flotante desbordan o se anulan con argumentos moderados. Este tema determina esos límites, introduce la función log-sum-exp y su evaluación estable, deduce de ella la forma correcta de calcular la entropía cruzada a partir de los logits y termina con la propagación de `NaN` y un procedimiento para diagnosticar una pérdida que deja de ser finita. Se sigue [9, §4.1], [26, §4.5.2] y las prácticas de CS336 [5].

6.1 Límites de la exponencial

Proposición 6.1. *En un formato con mayor número finito $x_{\max}$ y menor normal $x_{\min}$, el valor exacto e^t es finito y normal si y solo si $\ln x_{\min} \le t \le \ln x_{\max}$. Los intervalos son, aproximadamente,*

$$\textit{fp32 y bf16:}\ [-87{,}3;\ 88{,}7], \qquad \textit{fp16:}\ [-9{,}70;\ 11{,}09].$$

Por debajo del límite inferior, el resultado es subnormal, con menos bits de precisión, y algo más abajo se anula. Goodfellow *et al.* [9, §4.1] describen las dos formas en que falla la softmax ingenua $e^{z_i}/\sum_j e^{z_j}$: si algún logit es grande, el numerador y el denominador desbordan y el cociente ∞/∞ es `NaN`; si todos son muy negativos, ambos se anulan y $0/0$ es `NaN`.

6.2 La función log-sum-exp

Definición 6.2 (Log-sum-exp). Para $z \in \mathbb{R}^n$, $\mathrm{LSE}(z) = \ln \sum_{i=1}^{n} e^{z_i}$. La softmax del módulo 1 se escribe $\mathrm{softmax}(z)_i = e^{z_i - \mathrm{LSE}(z)}$.

Proposición 6.3 (Propiedades de log-sum-exp). *Sea $z \in \mathbb{R}^n$ y $c \in \mathbb{R}$.*

(i) $\mathrm{LSE}(z + c\mathbf{1}) = \mathrm{LSE}(z) + c$ *y* $\mathrm{softmax}(z + c\mathbf{1}) = \mathrm{softmax}(z)$.

(ii) $\max_i z_i \le \mathrm{LSE}(z) \le \max_i z_i + \ln n$*; la cota superior se alcanza si y solo si todas las componentes son iguales.*

(iii) $\nabla \mathrm{LSE}(z) = \mathrm{softmax}(z)$.

(iv) $\nabla^2 \mathrm{LSE}(z) = \mathrm{diag}(p) - pp^\top$ *con* $p = \mathrm{softmax}(z)$*, que es semidefinida positiva; en consecuencia,* LSE *es convexa.*

Proposición 6.4 (Evaluación estable). *Sea $c = \max_i z_i$. Entonces*

$$\mathrm{LSE}(z) = c + \ln S, \qquad \mathrm{softmax}(z)_i = \frac{e^{z_i - c}}{S}, \qquad S = \sum_{i=1}^{n} e^{z_i - c} \in [1, n].$$

En esta forma ninguna exponencial desborda, porque todos sus argumentos son ≤ 0, y el denominador nunca se anula, porque contiene el término $e^0 = 1$.

6.3 Entropía cruzada a partir de los logits

Proposición 6.6 (Log-softmax y entropía cruzada). *Sea $z \in \mathbb{R}^n$ el vector de logits y $y \in \{1, \ldots, n\}$ la clase correcta. Entonces*

$$\ln \mathrm{softmax}(z)_i = z_i - \mathrm{LSE}(z), \qquad \ell(z, y) = -\ln \mathrm{softmax}(z)_y = \mathrm{LSE}(z) - z_y \geq 0,$$

$$\nabla_z \ell = \mathrm{softmax}(z) - e_y,$$

donde e_y es el vector canónico.

La fórmula $\ell = \mathrm{LSE}(z) - z_y$, calculada con la proposición 6.4, no evalúa nunca el logaritmo de un número que pueda haberse anulado. Por eso las funciones de pérdida de las bibliotecas, como `F.cross_entropy` de PyTorch, reciben logits y no probabilidades [26, §4.5.2], y la composición `torch.log(torch.softmax(z))` debe evitarse.

6.4 Propagación de `NaN` y diagnóstico

La norma IEEE 754 [14] define como `NaN` los resultados de las operaciones inválidas $\infty - \infty$, $0 \cdot \infty$, $0/0$, ∞/∞ y $\sqrt{x}$ con $x < 0$, y establece que toda operación aritmética con un operando `NaN` da `NaN`; además, toda comparación con `NaN` es falsa. En particular, $\texttt{NaN} \cdot 0 = \texttt{NaN}$: multiplicar por cero no limpia el valor.

Observación 6.8 (Contagio). Si la pérdida de un paso es `NaN`, la retropropagación multiplica ese valor por todos los jacobianos locales, de modo que el gradiente de todo parámetro del que depende la pérdida es `NaN`, y la actualización convierte esos parámetros en `NaN`. A partir de ese paso la red ya no produce números. Por eso el diagnóstico debe hacerse en el primer paso en que aparece un valor no finito.

Observación 6.9 (Lo que se observa en la práctica). En la práctica del módulo se provoca el fallo a propósito: una red residual sin normalizar entrenada con tasa de aprendizaje 10 pasa de una pérdida de 2,6 a 10^{30} y llega a `NaN` en el paso 24. Con recorte del gradiente a norma 1, la pérdida se mantiene finita durante los 60 pasos.

Síntoma	Causa probable	Remedio
La pérdida crece varios órdenes de magnitud y luego da `NaN`	tasa de aprendizaje alta; gradientes que explotan (tema 1)	bajar la tasa, calentamiento, recorte del gradiente (tema 4)
`inf` repentino en fp16	desbordamiento por encima de 65 504	bf16, o escalado dinámico de la pérdida (tema 5)
`NaN` en una softmax o un logaritmo	exponencial desbordada, $\ln 0$	proposición 6.4; entropía cruzada con logits
`NaN` en una normalización	división por $\mathrm{RMS}(x) = 0$	$\epsilon > 0$ dentro de la raíz (tema 3)
Picos aislados de la pérdida	lote anómalo	recorte, descartar el lote, volver a un punto de control

Cuadro 6.1: Diagnóstico de una pérdida no finita.

Bibliografía

[1] J. L. Ba, J. R. Kiros y G. E. Hinton. Layer normalization. Prepublicación, 2016. https://arxiv.org/abs/1607.06450

[2] C. M. Bishop y H. Bishop. *Deep Learning: Foundations and Concepts*. Springer, 2024. Cap. 7, «Gradient Descent» (§7.4 «Normalization»); cap. 9, «Regularization» (§9.5 «Residual Connections» y §9.6.1 «Dropout»). https://www.bishopbook.com/

[3] F.-F. Li *et al.* CS231n: Deep Learning for Computer Vision. Notas del curso. Stanford University, 2026. «Neural Networks Part 2: Setting up the Data and the Loss», apartados «Weight Initialization» y «Regularization». https://cs231n.github.io/neural-networks-2/

[4] C. Manning *et al.* CS224N: Natural Language Processing with Deep Learning. Stanford University, invierno de 2021. Práctica 3, «Dependency Parsing». https://web.stanford.edu/class/archive/cs/cs224n/cs224n.1214/assignments/a3.pdf

[5] T. Hashimoto y P. Liang. CS336: Language Modeling from Scratch. Stanford University, primavera de 2026. Práctica 1, «Basics», versión 26.0.3. https://github.com/stanford-cs336/assignment1-basics

[6] T. Hashimoto y P. Liang. CS336: Language Modeling from Scratch. Stanford University, primavera de 2026. Práctica 2, «Systems», versión 26.1.3. https://github.com/stanford-cs336/assignment2-systems

[7] X. Glorot e Y. Bengio. Understanding the difficulty of training deep feedforward neural networks. En *Proceedings of the 13th International Conference on Artificial Intelligence and Statistics (AISTATS)*, PMLR 9:249–256, 2010.

[8] D. Goldberg. What every computer scientist should know about floating-point arithmetic. *ACM Computing Surveys*, 23(1):5–48, 1991.

[9] I. Goodfellow, Y. Bengio y A. Courville. *Deep Learning*. MIT Press, 2016. Cap. 4, «Numerical Computation»; cap. 7, «Regularization for Deep Learning»; cap. 8, «Optimization for Training Deep Models»; cap. 10, «Sequence Modeling: Recurrent and Recursive Nets». https://www.deeplearningbook.org/

[10] K. He, X. Zhang, S. Ren y J. Sun. Deep residual learning for image recognition. En *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, págs. 770–778, 2016. https://arxiv.org/abs/1512.03385

[11] K. He, X. Zhang, S. Ren y J. Sun. Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification. En *IEEE International Conference on Computer Vision (ICCV)*, 2015. https://arxiv.org/abs/1502.01852

[12] K. He, X. Zhang, S. Ren y J. Sun. Identity mappings in deep residual networks. En *European Conference on Computer Vision (ECCV)*, 2016. https://arxiv.org/abs/1603.05027

[13] N. J. Higham. *Accuracy and Stability of Numerical Algorithms*, 2.ª ed. SIAM, 2002.

[14] IEEE Computer Society. *IEEE Standard for Floating-Point Arithmetic*, IEEE Std 754-2019. IEEE, 2019.

[15] S. Ioffe y C. Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. En *International Conference on Machine Learning (ICML)*, 2015. https://arxiv.org/abs/1502.03167

[16] P. Micikevicius, S. Narang, J. Alben, G. Diamos, E. Elsen, D. Garcia, B. Ginsburg, M. Houston, O. Kuchaiev, G. Venkatesh y H. Wu. Mixed precision training. En *International Conference on Learning Representations (ICLR)*, 2018. https://arxiv.org/abs/1710.03740

[17] P. Micikevicius *et al.* FP8 formats for deep learning. Prepublicación, 2022. https://arxiv.org/abs/2209.05433

[18] R. Pascanu, T. Mikolov e Y. Bengio. On the difficulty of training recurrent neural networks. En *International Conference on Machine Learning (ICML)*, PMLR 28(3):1310–1318, 2013. https://arxiv.org/abs/1211.5063

[19] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei e I. Sutskever. Language models are unsupervised multitask learners. Informe técnico, OpenAI, 2019.

[20] B. Raj *et al.* 11-785: Introduction to Deep Learning. Carnegie Mellon University, otoño de 2025. https://deeplearning.cs.cmu.edu/

[21] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever y R. Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(56):1929–1958, 2014. https://jmlr.org/papers/v15/srivastava14a.html

[22] H. Touvron *et al.* LLaMA: Open and efficient foundation language models. Prepublicación, 2023. https://arxiv.org/abs/2302.13971

[23] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser e I. Polosukhin. Attention is all you need. En *Advances in Neural Information Processing Systems 30 (NeurIPS)*, 2017. https://arxiv.org/abs/1706.03762

[24] xAI. Grok-1: código y pesos publicados. 2024. https://github.com/xai-org/grok-1

[25] R. Xiong, Y. Yang, D. He, K. Zheng, S. Zheng, C. Xing, H. Zhang, Y. Lan, L. Wang y T.-Y. Liu. On layer normalization in the Transformer architecture. En *International Conference on Machine Learning (ICML)*, 2020. https://arxiv.org/abs/2002.04745

[26] A. Zhang, Z. C. Lipton, M. Li y A. J. Smola. *Dive into Deep Learning*, versión 1.0.3. Cambridge University Press, 2023. Cap. 4, «Linear Neural Networks for Classification»; cap. 5, «Multilayer Perceptrons»; cap. 8, «Modern Convolutional Neural Networks»; cap. 9, «Recurrent Neural Networks». https://d2l.ai/

[27] B. Zhang y R. Sennrich. Root mean square layer normalization. En *Advances in Neural Information Processing Systems 32 (NeurIPS)*, 2019. https://arxiv.org/abs/1910.07467